

SQL MASTERY

Lesson 9 — JOINS

MySQL • MySQL Workbench • Placement & Interview Preparation

Goal: Learn how to combine data from multiple tables using JOINS and recognize common placement-question patterns.

1. What is a JOIN?

A JOIN combines rows from two or more tables using a related column. Relational databases usually split information across tables, so JOINS let us bring related information together.

Example: A students table may contain *dept_id*, while a departments table contains the department name. A JOIN connects the two using *dept_id*.

2. Practice Tables

```
CREATE TABLE students (  
  student_id INT PRIMARY KEY,  
  name VARCHAR(50),  
  dept_id INT  
);
```

```
CREATE TABLE departments (  
  dept_id INT PRIMARY KEY,  
  department_name VARCHAR(50)  
);
```

```
INSERT INTO departments VALUES  
(1, 'Computer Science'),  
(2, 'Mechanical'),  
(3, 'Electronics'),  
(5, 'Civil');
```

```
INSERT INTO students VALUES  
(101, 'Arun', 1),  
(102, 'Priya', 2),  
(103, 'Rahul', 1),  
(104, 'Sneha', 3),  
(105, 'Divya', 4),  
(106, 'Vijay', NULL);
```

Important: Divya has dept_id 4, which does not exist in departments. Vijay has NULL. Civil exists in departments but has no student. These unmatched rows make the JOIN behavior easy to see.

3. INNER JOIN

Concept: INNER JOIN returns only rows that have a matching value in both tables.

```
SELECT s.name, d.department_name  
FROM students s  
INNER JOIN departments d  
ON s.dept_id = d.dept_id;
```

Result: Arun–Computer Science, Priya–Mechanical, Rahul–Computer Science, Sneha–Electronics. Divya and Vijay are excluded because they have no matching department.

Remember: INNER JOIN = **matching rows only**.

4. LEFT JOIN

LEFT JOIN returns every row from the left table and matching rows from the right table. If there is no match, columns from the right table become NULL.

```
SELECT s.name, d.department_name
FROM students s
LEFT JOIN departments d
ON s.dept_id = d.dept_id;
```

Here every student appears. Divya and Vijay have NULL for department.

Remember: LEFT JOIN = **all left rows + matching right rows.**

5. RIGHT JOIN

RIGHT JOIN returns every row from the right table and matching rows from the left table.

```
SELECT s.name, d.department_name
FROM students s
RIGHT JOIN departments d
ON s.dept_id = d.dept_id;
```

Civil appears with NULL for student because Civil has no matching student.

Remember: RIGHT JOIN = **all right rows + matching left rows.**

6. FULL OUTER JOIN in MySQL

MySQL does not provide FULL OUTER JOIN as a direct JOIN keyword. A common workaround is LEFT JOIN UNION RIGHT JOIN.

```
SELECT s.name, d.department_name
FROM students s
LEFT JOIN departments d
ON s.dept_id = d.dept_id

UNION

SELECT s.name, d.department_name
FROM students s
RIGHT JOIN departments d
ON s.dept_id = d.dept_id;
```

This keeps unmatched rows from both sides as well as matching rows.

7. JOIN Aliases

Aliases make JOIN queries shorter and easier to read.

```
SELECT s.name, d.department_name
FROM students s
INNER JOIN departments d
ON s.dept_id = d.dept_id;
```

Here **s** represents students and **d** represents departments.

8. JOIN + WHERE

Use WHERE to filter rows after the tables have been joined.

```
SELECT s.name, d.department_name
FROM students s
INNER JOIN departments d
ON s.dept_id = d.dept_id
WHERE d.department_name = 'Computer Science';
```

This returns only students in Computer Science.

9. JOIN + ORDER BY

```
SELECT s.name, d.department_name
FROM students s
INNER JOIN departments d
ON s.dept_id = d.dept_id
ORDER BY d.department_name ASC;
```

JOIN combines the data; ORDER BY sorts the resulting rows.

10. JOIN + GROUP BY

A common interview question is: count students in every department, including departments with zero students.

```
SELECT d.department_name,
COUNT(s.student_id) AS total_students
FROM departments d
LEFT JOIN students s
ON d.dept_id = s.dept_id
GROUP BY d.department_name;
```

Civil appears with count 0 because LEFT JOIN preserves all departments.

Important: COUNT(s.student_id) is preferable here to COUNT(*) because the unmatched row has NULL student_id, so it contributes zero to COUNT(column).

11. JOIN + HAVING

```
SELECT d.department_name,
COUNT(s.student_id) AS total_students
FROM departments d
LEFT JOIN students s
ON d.dept_id = s.dept_id
GROUP BY d.department_name
HAVING COUNT(s.student_id) > 1;
```

This keeps only departments having more than one student.

12. SELF JOIN

A SELF JOIN joins a table with itself. It is useful for employee-manager relationships, category hierarchies, and similar parent-child data.

```
CREATE TABLE employee_manager (
emp_id INT PRIMARY KEY,
emp_name VARCHAR(50),
manager_id INT
);

INSERT INTO employee_manager VALUES
(1, 'Amit', NULL),
(2, 'Rahul', 1),
(3, 'Sneha', 1),
(4, 'Priya', 2),
(5, 'Vijay', 2);

SELECT
e.emp_name AS Employee,
m.emp_name AS Manager
FROM employee_manager e
LEFT JOIN employee_manager m
ON e.manager_id = m.emp_id;
```

The same table is treated as two logical tables: **e** for employee and **m** for manager.

13. JOIN Types — Quick Comparison

JOIN	What it returns
INNER JOIN	Only matching rows from both tables
LEFT JOIN	All left rows + matching right rows
RIGHT JOIN	All right rows + matching left rows
FULL OUTER JOIN	All rows from both sides; MySQL uses a workaround
SELF JOIN	A table joined to itself

14. SQL Execution Order with JOIN

FROM
 ↓
 JOIN
 ↓
 ON
 ↓
 WHERE
 ↓
 GROUP BY
 ↓
 HAVING
 ↓
 SELECT
 ↓
 ORDER BY
 ↓
 LIMIT

This order helps explain why WHERE filters rows while HAVING filters groups.

15. WHERE vs HAVING

WHERE	HAVING
Filters individual rows	Filters groups
Usually before GROUP BY	After GROUP BY
Example: salary > 50000	Example: COUNT(*) > 2
Generally not used for aggregate results	Designed for aggregate/group conditions

16. Common Placement Patterns

Question wording	Likely concept
Employee with department	INNER JOIN
All customers, even those without orders	LEFT JOIN
Departments with no employees	LEFT JOIN + IS NULL or COUNT
Employee and manager	SELF JOIN
Count employees in each department	JOIN + GROUP BY
Groups with count greater than N	GROUP BY + HAVING

17. Common Mistakes

1. Forgetting the ON condition can produce incorrect results or a Cartesian product.
2. Using INNER JOIN when unmatched rows must be preserved.
3. Using WHERE with COUNT/AVG/SUM when HAVING is required.
4. Confusing the left and right table. In LEFT JOIN, the table written after FROM is the left table.
5. Using COUNT(*) when you actually need to count only matched rows in a LEFT JOIN.

18. Interview Tips

- If the question says **only matching**, think INNER JOIN.
- If it says **all customers/students/employees even if...**, think LEFT JOIN.
- If it says **no matching record**, think LEFT JOIN + IS NULL.
- If it says **each department/category**, consider GROUP BY.
- If it says **departments whose average/count/total...**, consider HAVING.
- Prefer LEFT JOIN in many practical queries because it clearly communicates which table must be preserved.

19. Practice Questions

1. Display student name and department name.
2. Display all students, including students whose department does not exist.
3. Display all departments, including departments with no students.
4. Display students belonging to Mechanical.
5. Count students in each department.
6. Display departments having more than one student.
7. Display departments having zero students.
8. Display students who do not have a matching department.
9. Display employee and manager names using SELF JOIN.
10. Display departments ordered by number of students from highest to lowest.

20. Key Takeaways

INNER JOIN → matching rows only

LEFT JOIN → all left rows + matches

RIGHT JOIN → all right rows + matches

FULL OUTER JOIN → all rows from both sides; use UNION in MySQL

SELF JOIN → table joined with itself

JOIN + GROUP BY → combine tables and aggregate

JOIN + HAVING → filter aggregated groups

Next Lesson

Lesson 10 — Subqueries & Nested Queries

Scalar subqueries • IN • EXISTS • NOT EXISTS • Correlated subqueries • Placement-level problems